Instruction For Reproducing Section 4.1 Under PxG

Covariate-Assisted Bayesian Graph Learning for Heterogeneous Data

Description: This R Markdown file provides the workflow for reproducing simulation results in §4.1 under the PxG model.

1. Generating Simulation Data

```
rm(list = ls())

## loading libraries
library(huge)
library(invgamma)
library(BDgraph)

## reading functions for the simulation study
## "PxG-GW.R" contains all functions for Gaussian-G-Wishart model (section 3.2.1)
source("PxG-GW.R")

## generating simulation data from the 3-dimensional GGM in section 4.1
## first, generating covariate x
set.seed(2020)
x = rep(0, 300)
x[1:100] = runif(100, -1, -1/3)
x[101:200] = runif(100, -1/3, 1/3)
x[201:300] = runif(100, 1/3, 1)
x = x[order(x)]

## true precision matrix as a piecewise function of x
Omega = function(i, x){
  Omega = diag(3)*1.2
  if(i == 1){
    Omega[1,2] = Omega[2,1] = 0
    Omega[1,3] = Omega[3,1] = -0.75*x+0.25
    Omega[2,3] = Omega[3,2] = 0.75*x+1.25
  }
  if(i == 2){
    Omega[1,2] = Omega[2,1] = 0.75*x+0.75
    Omega[1,3] = Omega[3,1] = 0
    Omega[2,3] = Omega[3,2] = -0.75*x+0.75
  }
  if(i == 3){
    Omega[1,2] = Omega[2,1] = -0.75*x+1.25
    Omega[1,3] = Omega[3,1] = 0.75*x+0.25
    Omega[2,3] = Omega[3,2] = 0
  }
  return(Omega)
```

```
}

## true graph as a function of x
G0 = function(i, x){
  Omega = diag(3)
  if(i == 1){
    Omega[1,2] = Omega[2,1] = 0
    Omega[1,3] = Omega[3,1] = 1
    Omega[2,3] = Omega[3,2] = 1
  }
  if(i == 2){
    Omega[1,2] = Omega[2,1] = 1
    Omega[1,3] = Omega[3,1] = 0
    Omega[2,3] = Omega[3,2] = 1
  }
  if(i == 3){
    Omega[1,2] = Omega[2,1] = 1
    Omega[1,3] = Omega[3,1] = 1
    Omega[2,3] = Omega[3,2] = 0
  }
  return(Omega)
}

## generating response y according to the true precision matrix
y = matrix(0, 300, 3)
for(i in 1:100){
  temp = solve(Omega(1, x[i]))
  y[i,] = t(t(chol(temp))%*%rnorm(3,0,1))
}
for(i in 101:200){
  temp = solve(Omega(2, x[i]))
  y[i,] = t(t(chol(temp))%*%rnorm(3,0,1))
}
for(i in 201:300){
  temp = solve(Omega(3, x[i]))
  y[i,] = t(t(chol(temp))%*%rnorm(3,0,1))
}

x = matrix(x, 300, 1)
Y = apply(y, 2, function(x) (x-mean(x))/sd(x))
X = apply(x, 2, function(x) (x-mean(x))/sd(x))
```

2. Fitting Three Models: PxG, Covariate-only, Graph-only. Comparing DIC Values.

```
## fitting simulation data with the proposed PxG model
set.seed(2020)
A = PxG_GW(Y = Y, X = X, b = 3, D = diag(3), alphaG = 0.1,
           mu0 = as.vector(rep(0,1)), sigsq0 = 1, b1 = 1, b2 = 1, M = 1, N = 5,
           burnin = 500, nmcmc = 1000, z0 = sample(1:15, 300, TRUE, rep(1/15, 15)), B = 1)

## fitting simulation data with the covariate-only model
set.seed(2020)
```

```
B = CovOnly_GW(Y = Y, X = X, b = 3, D = diag(3), alphaG = 0.1,
               mu0 = as.vector(rep(0,1)), sigsq0 = 1, b1 = 1, b2 = 1, M = 1, N = 5,
               burnin = 500, nmcmc = 1000, z0 = sample(1:15, 300, TRUE, rep(1/15, 15)))

## fitting simulation data with the graph-only model
set.seed(2020)
C = GraphOnly_GW(Y = Y, X = X, b = 3, D = diag(3), alphaG = 0.1,
                 mu0 = as.vector(rep(0,1)), sigsq0 = 1, b1 = 1, b2 = 1, M = 1, N = 5,
        burnin = 500, nmcmc = 1000, z0 = sample(1:15, 300, TRUE, rep(1/15, 15)), B = 1)
```

```
# DIC for the PxG model
A$DIC
```

```
## [1] 2193.304
```

```
# DIC for the covariate-only model
B$DIC
```

```
## [1] 2584.573
```

```
# DIC for the graph-only model
C$DIC
```

```
## [1] 3189.946
```

3. Preparation For Plots

```
## generating true edge and partial correlation sequences for plot purposes
xseq = seq(-1, 1, by = 0.01)
Gseq = Oseq = array(0, c(3,3,length(xseq)))
for(i in 1:length(xseq)){
  if(xseq[i]<(-1/3)) { Oseq[,,i] = Omega(1, xseq[i]); Gseq[,,i] = G0(1, xseq[i]) }
  if(xseq[i]>1/3)    { Oseq[,,i] = Omega(3, xseq[i]); Gseq[,,i] = G0(3, xseq[i]) }
  if((xseq[i]>-1/3)&&(xseq[i]<1/3))
  { Oseq[,,i] = Omega(2, xseq[i]); Gseq[,,i] = G0(2, xseq[i]) }
}

## true partial correlation sequences
w12 = Oseq[1,2,]/1.2
w13 = Oseq[1,3,]/1.2
w23 = Oseq[2,3,]/1.2

## true edge sequences
e12 = Gseq[1,2,]
e13 = Gseq[1,3,]
e23 = Gseq[2,3,]

## calculate true partial correlations for each covariate sample
xw12 = xw13 = xw23 = rep(0, 300)
for(i in 1:100){
  temp = Omega(1, x[i])
  xw12[i] = temp[1,2]/1.2
  xw13[i] = temp[1,3]/1.2
  xw23[i] = temp[2,3]/1.2
}
for(i in 101:200){
```

```
  temp = Omega(2, x[i])
  xw12[i] = temp[1,2]/1.2
  xw13[i] = temp[1,3]/1.2
  xw23[i] = temp[2,3]/1.2
}
for(i in 201:300){
  temp = Omega(3, x[i])
  xw12[i] = temp[1,2]/1.2
  xw13[i] = temp[1,3]/1.2
  xw23[i] = temp[2,3]/1.2
}

## loading libraries for plot
library(ggplot2)
library(ggpubr)
```

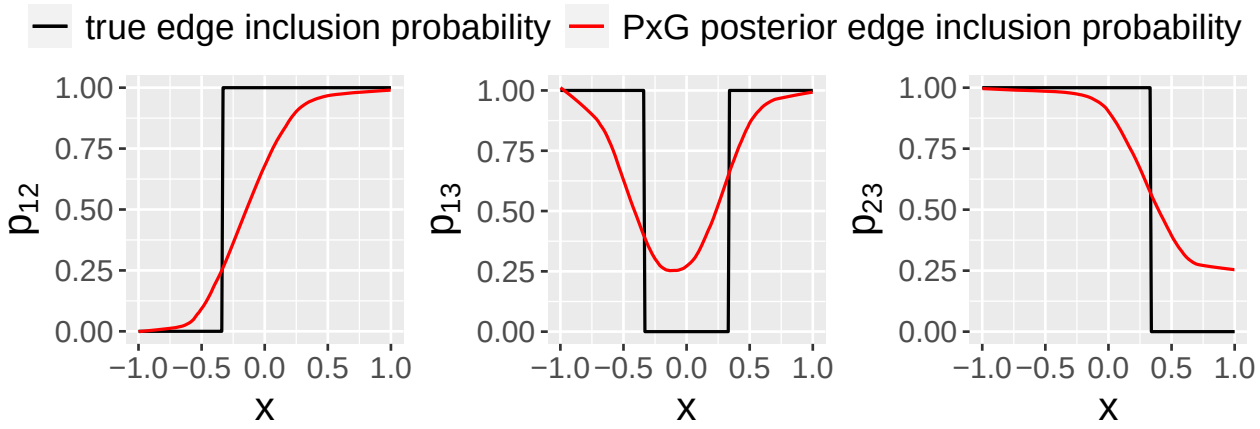
4. Generating Figure 3

```
## plot Figure 3
a = A$Gsum[1,2,]
temp1 = data.frame(x1 = xseq, y1 = e12)
post = lowess(x, a, f = 0.3)
temp2 = data.frame(x2 = post$x, y2=post$y)
p1 = ggplot() +
  geom_line(temp1, mapping = aes(x=x1,y=y1,
            color = "true edge inclusion probability"), size = 0.6) +
  geom_line(temp2, mapping = aes(x=x2,y=y2,
            color = "PxG posterior edge inclusion probability"), size = 0.6) +
  scale_colour_manual(values = c("true edge inclusion probability" = "black",
                                 "PxG posterior edge inclusion probability" = "red")) +
  xlab("x") + ylab(expression(p[12])) + labs(color = "") +
  theme(axis.text = element_text(size = 12),
        axis.title.x = element_text(size = 16),
        axis.title.y = element_text(size = 16,face = "bold"),
        legend.text = element_text(size = 14))

a = A$Gsum[1,3,]
temp1 = data.frame(x1=xseq, y1=e13)
post = lowess(x, a, f=0.3)
temp2 = data.frame(x2=post$x, y2=post$y)
p2 = ggplot() +
  geom_line(temp1, mapping=aes(x=x1,y=y1,
            color = "true edge inclusion probability"), size = 0.6) +
  geom_line(temp2, mapping=aes(x=x2,y=y2,
            color = "PxG posterior edge inclusion probability"), size = 0.6) +
  scale_colour_manual(values = c("true edge inclusion probability" = "black",
                                 "PxG posterior edge inclusion probability" = "red")) +
  xlab("x") + ylab(expression(p[13])) + labs(color = "") +
  theme(axis.text=element_text(size=12),
        axis.title.x=element_text(size=16),
        axis.title.y=element_text(size=16,face="bold"),
        legend.text = element_text(size=14))
```

```
a = A$Gsum[2,3,]
temp1 = data.frame(x1=xseq, y1=e23)
post = lowess(x, a, f=0.3)
temp2 = data.frame(x2=post$x, y2=post$y)
p3 = ggplot() +
  geom_line(temp1, mapping=aes(x=x1,y=y1,
            color = "true edge inclusion probability"), size = 0.6) +
  geom_line(temp2, mapping=aes(x=x2,y=y2,
            color = " PxG posterior edge inclusion probability"), size = 0.6) +
  scale_colour_manual(values = c("true edge inclusion probability" = "black",
                                 " PxG posterior edge inclusion probability" = "red")) +
  xlab("x") + ylab(expression(p[23])) + labs(color = "") +
  theme(axis.text=element_text(size=12),
        axis.title.x=element_text(size=16),
        axis.title.y=element_text(size=16,face="bold"),
        legend.text = element_text(size=14))

p0 = ggarrange(p1, p2, p3, ncol = 3, nrow = 1, common.legend = TRUE); p0
```



5. Generating Figure 4 and Display MSE

```
## plot Figure 4 and calculate MSE
a = rep(0, length(x))
for(i in 1:length(x)) a[i] = A$Osum[1,2,i]/sqrt(A$Osum[1,1,i]*A$Osum[2,2,i])
# MSE
mean((a - xw12)^2)
```

```
## [1] 0.02505988
```

```
temp1 = data.frame(x1=xseq, y1=w12)
post = lowess(x, a, f=0.3)
temp2 = data.frame(x2=post$x, y2=post$y)
p1 = ggplot() +
  geom_line(temp1, mapping=aes(x=x1,y=y1,
                               color = "true partial correlation"), size = 0.6) +
  geom_line(temp2, mapping=aes(x=x2,y=y2,
                               color = "PxG posterior estimate"), size = 0.6) +
  scale_colour_manual(values = c("true partial correlation" = "black",
                                 "PxG posterior estimate" = "red")) +
```

```
  xlab("x") + ylab(expression(rho[12])) + labs(color = "") +
  theme(axis.text=element_text(size=12),
        axis.title.x=element_text(size=16),
        axis.title.y=element_text(size=16,face="bold"),
        legend.text = element_text(size=14)) +
  annotate("text", x = -0.5, y = 0.8, label = "MSE") +
  annotate("text", x = -0.5, y = 0.7, label = "PxG  0.0251")

a = rep(0, length(x))
for(i in 1:length(x)) a[i] = A$Osum[1,3,i]/sqrt(A$Osum[1,1,i]*A$Osum[3,3,i])
# MSE
mean((a - xw13)^2)
```

```
## [1] 0.02206017
```

```
temp1 = data.frame(x1=xseq, y1=w13)
post = lowess(x, a, f=0.3)
temp2 = data.frame(x2=post$x, y2=post$y)
p2 = ggplot() +
  geom_line(temp1, mapping=aes(x=x1,y=y1,
                               color = "true partial correlation"), size = 0.6) +
  geom_line(temp2, mapping=aes(x=x2,y=y2,
                               color = "PxG posterior estimate"), size = 0.6) +
  scale_colour_manual(values = c("true partial correlation" = "black",
                                 "PxG posterior estimate" = "red")) +
  xlab("x") + ylab(expression(rho[13])) + labs(color = "") +
  theme(axis.text=element_text(size=12),
        axis.title.x=element_text(size=16),
        axis.title.y=element_text(size=16,face="bold"),
        legend.text = element_text(size=14)) +
  annotate("text", x = 0, y = 0.855, label = "MSE") +
  annotate("text", x = 0, y = 0.755, label = "PxG  0.0221")

a = rep(0, length(x))
for(i in 1:length(x)) a[i] = A$Osum[2,3,i]/sqrt(A$Osum[2,2,i]*A$Osum[3,3,i])
# MSE
mean((a - xw23)^2)
```

```
## [1] 0.01041099
```

```
temp1 = data.frame(x1=xseq, y1=w23)
post = lowess(x, a, f=0.3)
temp2 = data.frame(x2=post$x, y2=post$y)
p3 = ggplot() +
  geom_line(temp1, mapping=aes(x=x1,y=y1,
                               color = "true partial correlation"), size = 0.6) +
  geom_line(temp2, mapping=aes(x=x2,y=y2,
                               color = "PxG posterior estimate"), size = 0.6) +
  scale_colour_manual(values = c("true partial correlation" = "black",
                                 "PxG posterior estimate" = "red")) +
  xlab("x") + ylab(expression(rho[23])) + labs(color = "") +
  theme(axis.text=element_text(size=12),
        axis.title.x=element_text(size=16),
        axis.title.y=element_text(size=16,face="bold"),
        legend.text = element_text(size=14)) +
```

```
  annotate("text", x = 0.5, y = 0.8, label = "MSE") +
  annotate("text", x = 0.5, y = 0.7, label = "PxG  0.0104")

p0 = ggarrange(p1, p2, p3, ncol = 3, nrow = 1, common.legend = TRUE); p0
```

